# Time-Marching Physics-Informed Neural Networks for the Planar Newtonian Three-Body Problem

[Author names]
Department of [Department], [University]

**Abstract**

We study whether a physics-informed neural network (PINN) can represent the trajectories of the planar Newtonian three-body problem without any trajectory data. The network is trained only on the equations of motion. Two design choices make this work. First, the initial state is built into the ansatz, $x(t) = x_0 + v_0 t + t^2 N_\theta(t)$, so the initial conditions hold exactly. Second, the time interval is split into segments, each with its own network and started from the state predicted at the end of the previous segment (time marching). On the Chenciner–Montgomery figure-eight orbit, twelve segments trained for 4000 Adam steps each reproduce the high-accuracy DOP853 reference over one period to a maximum position error of $3.3 \times 10^{-4}$, with a relative energy drift of $1.3 \times 10^{-4}$. The same method gives a maximum error of $3.0 \times 10^{-5}$ on the equal-mass Lagrange triangle and $5.9 \times 10^{-4}$ on a random, non-periodic initial condition over a horizon of $T = 2$. Monolithic PINNs with the same ansatz or with a soft initial-condition penalty fail on the figure-eight (errors 3.6 and 2.5), and a supervised MLP trained on 200 reference samples reaches $2.0 \times 10^{-2}$. The method does not extrapolate: beyond the trained window the error grows to order 10. We discuss this limitation and the cost of training.

## 1 Introduction

The planar three-body problem asks for the motion of three point masses attracted to one another by Newtonian gravity. Except for special cases, it has no closed-form solution, and for generic initial conditions the motion is chaotic [1]. Numerical integration remains the standard tool, and high-order integrators such as the Dormand–Prince 8(5,3) method (DOP853) [3] reach near machine precision over one orbit. The problem is nevertheless a standard test case for new methods, because it combines a small system with rich dynamics.

Physics-informed neural networks (PINNs) [5] approximate the solution of a differential equation by a neural network, with the equation residual at collocation points as the training loss. PINNs have been applied to many ODE and PDE problems, but they are known to fail to train on some problems, particularly over long time intervals and on oscillatory solutions [6, 7, 8]. Long-horizon ODE trajectories are a known difficulty: a residual loss constrains the solution only locally, and errors can accumulate in time.

This article asks whether a PINN can reproduce the three-body trajectories to a precision useful for scientific work. The contributions are: (i) a PINN formulation for the planar three-body problem that has no trajectory data and builds the initial state into the ansatz; (ii) a time-marching variant that trains one network per time segment, and a study of how segment count and training budget affect accuracy; (iii) a validation against a high-accuracy reference on a periodic figure-eight orbit, a second periodic orbit, a non-periodic random initial condition, and two baseline methods; and (iv) an explicit account of where the method fails.

## 2 Problem and reference solution

Consider three bodies with masses $m_1, m_2, m_3$ in the plane, with $G = 1$. Let $x_i(t) \in \mathbb{R}^2$ be the position of body $i$. Newton's equations are

$$\ddot{x}_i(t) = \sum_{j \neq i} m_j \, \frac{x_j(t) - x_i(t)}{\|x_j(t) - x_i(t)\|^3}, \qquad i = 1, 2, 3, \tag{1}$$

with initial state $x_i(0) = x_i^0$ and $\dot{x}_i(0) = v_i^0$. The total energy $E = \frac{1}{2}\sum_i m_i \|\dot{x}_i\|^2 - \sum_{i<j} m_i m_j / \|x_i - x_j\|$ and the angular momentum $L = \sum_i m_i \, (x_i \times \dot{x}_i)$ are conserved.

The reference solution is computed with the Dormand–Prince 8(5,3) method (DOP853, SciPy [4]) with relative and absolute tolerances of $10^{-12}$. For the figure-eight orbit, the integrator conserves $E$ and $L$ to $2 \times 10^{-11}$ over one period, and the solution returns to its initial state to $4 \times 10^{-8}$; the residual difference comes from the eight-digit precision of the published initial condition. The Lagrange triangle closes to $8 \times 10^{-12}$ over one period and conserves $E$ to $10^{-11}$.

## 3 Method

### 3.1 Hard initial-condition ansatz

A network $N_\theta : \mathbb{R} \to \mathbb{R}^6$ with tanh activations is used to write the trajectory as

$$x_\theta(t) = x^0 + v^0 \, t + t^2 \, N_\theta(t). \tag{2}$$

Since $x_\theta(0) = x^0$ and $\dot{x}_\theta(0) = v^0$ hold identically, the initial state is never a loss term. The output of $N_\theta$ is initialised close to zero, so the untrained model is the free-motion line.

### 3.2 Residual loss

Derivatives of $x_\theta$ with respect to $t$ are obtained by forward-mode automatic differentiation (nested Jacobian–vector products), which is cheaper than the reverse-mode second derivatives used in standard PINN code and gives the same values (checked against central finite differences to $10^{-6}$ in our tests). The loss is the mean squared residual of Eq. (1) at $n_c$ collocation times drawn uniformly on the segment:

$$\mathcal{L}(\theta) = \frac{1}{n_c} \sum_{k=1}^{n_c} \sum_{i=1}^{3} \Big\| \ddot{x}_{\theta,i}(t_k) - \textstyle\sum_{j \neq i} m_j \, \frac{x_{\theta,j}(t_k) - x_{\theta,i}(t_k)}{\|x_{\theta,j}(t_k) - x_{\theta,i}(t_k)\|^3} \Big\|^2. \tag{3}$$

No reference data enter the loss.

### 3.3 Time marching

A single network over the full period must represent a trajectory whose residual constrains it only locally. We therefore split $[0, T]$ into $S$ segments of length $L = T/S$. Segment $k$ has its own network, trained on $[0, L]$ with initial state $y_k = (x, \dot{x})$ equal to the state predicted by segment $k - 1$ at its end time $t = L$. The handoff is evaluated exactly at $t = L$; an early version evaluated it at the last sample before $L$, which introduces an error of order $|\dot{x}| \, \Delta t$ at every boundary and caused a plateau that we initially misattributed to optimisation. The corrected handoff is what all reported results use. The segments are not coupled in the loss; continuity is enforced by construction through the handoff, and a trajectory evaluated across segments is continuous in $x$ and $\dot{x}$.

### 3.4 Training

Each segment is trained with Adam [9] (learning rate $10^{-3}$, cosine decay to $2 \times 10^{-5}$), width 128, depth 4, $n_c = 2048$ collocation points resampled at every step, and a fixed number of steps (4000 per segment). Networks and computation run in single precision on one NVIDIA RTX 4060 Ti; a float64 comparison on one 6-segment configuration, run before the handoff fix, changed the error by about 9%, so precision is not the limiting factor. An L-BFGS polish was tried in a preliminary run and did not improve accuracy, so it is not used.

## 4 Experiments

### 4.1 Test problems

- **Figure-eight** (Chenciner–Montgomery [2]): equal masses $m = 1$, $T = 6.32591398$. This is the main test.
- **Lagrange triangle**: equal masses on an equilateral rotating configuration, $T = 3.6276$. A second periodic orbit with a different geometry.
- **Random initial condition**: positions uniform in $[-1, 1]^2$, velocities $\mathcal{N}(0, 0.4^2)$, centre of mass at rest, drawn with seed 7 and accepted only if the reference trajectory keeps all pairwise distances above 0.2 on $[0, 2]$. This tests a non-periodic orbit.

Each PINN is evaluated on a dense grid of 4000 times over $[0, T]$, which was not used for training, against the DOP853 reference. We report the maximum and RMS position error, the maximum velocity error, the energy drift $\max_t |E_\theta(t) - E(0)|/|E(0)|$, where $E_\theta$ is computed from the network's positions and velocities, and the closure error $\|(x_\theta(T), \dot{x}_\theta(T)) - y^0\|_\infty$ for periodic orbits.

### 4.2 Baselines

1. **Monolithic hard-IC PINN**: the same ansatz as Eq. (2), one network over the full period, 20,000 Adam steps. This is fewer than the 48,000 steps used in total by the 12 segments, so the baseline is not given extra compute.
2. **Monolithic soft-IC PINN**: a network $x_\theta(t)$ with no ansatz, trained with the residual loss plus $10^3 \, \|x_\theta(0) - x^0\|^2 + \|\dot{x}_\theta(0) - v^0\|^2$, same budget.
3. **Supervised MLP**: width 128, depth 3, trained for 20,000 steps on 200 samples of the reference trajectory at random times in $[0, T]$. This baseline uses data, so it is not a like-for-like comparison, but it gives the accuracy that a data-driven method reaches with a modest number of samples.

## 5 Results

### 5.1 Figure-eight

Table 1 gives the main results. Figure 1 shows the trajectory, and Figure 2 shows the error against time for the PINN and the baselines.

The time-marching PINN is the only method with a per-period error below $10^{-3}$. Both monolithic PINNs fail to represent the orbit: their errors are of order one, comparable to the size of the orbit. The supervised MLP, using 200 exact samples, is about 60 times less accurate than the time-marching PINN.

Table 1: Accuracy on one period $[0, T]$ unless noted. Errors are absolute in the units $G = m = 1$.

| Method | max $\lvert x - x_{\text{ref}}\rvert$ | RMS $\lvert x - x_{\text{ref}}\rvert$ | max $\lvert E - E_0\rvert/\lvert E_0\rvert$ | time (s) |
|---|---|---|---|---|
| Time-marching PINN, figure-eight ($S$=12, 4000 steps) | $3.3 \times 10^{-4}$ | $7.1 \times 10^{-5}$ | $1.3 \times 10^{-4}$ | 724 |
| Monolithic hard-IC PINN (20k steps) | 3.63 | 1.53 | – | 318 |
| Monolithic soft-IC PINN (20k steps) | 2.47 | 0.99 | – | 423 |
| Supervised MLP (200 samples) | $2.0 \times 10^{-2}$ | $1.0 \times 10^{-2}$ | – | 15 |
| Time-marching PINN, Lagrange triangle | $3.0 \times 10^{-5}$ | $5.8 \times 10^{-6}$ | $5.1 \times 10^{-5}$ | 722 |
| Time-marching PINN, random IC ($S$=4, $T$=2) | $5.9 \times 10^{-4}$ | – | $5.6 \times 10^{-4}$ | 177 |

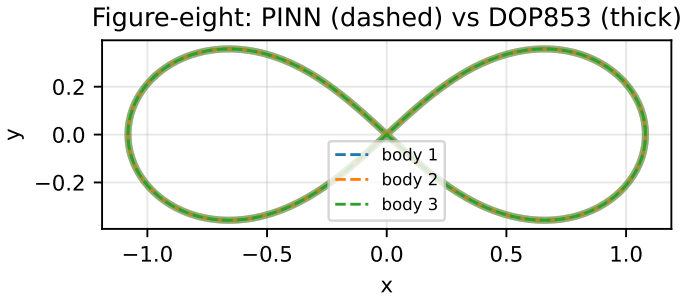



Figure 1: Figure-eight orbit. Dashed: PINN (12 segments). Thick: DOP853 reference. The curves are visually indistinguishable at this scale.

### 5.2 Segment count and training budget

Table 2 shows the effect of the number of segments and the Adam budget per segment, on the figure-eight over one period. These runs use a common seed and were run before the final configuration; they are included as a convergence study.

Table 2: Convergence study on the figure-eight (one period).

| Segments $S$ | Adam steps / segment | max $\lvert x - x_{\text{ref}}\rvert$ |
|---|---|---|
| 3 | 4000 | $1.09 \times 10^{-1}$ |
| 6 | 4000 | $1.21 \times 10^{-3}$ |
| 12 | 2000 | $5.31 \times 10^{-4}$ |
| 12 | 4000 | $1.66 \times 10^{-4}$ |

Shorter segments help more than longer training: three segments of length 2.1 fail, while twelve segments of length 0.53 reach $10^{-4}$. The final run in Table 1 uses per-segment seeds and gives $3.3 \times 10^{-4}$, so the run-to-run variation of this configuration is at least a factor of two. Figure 4 shows the same data.

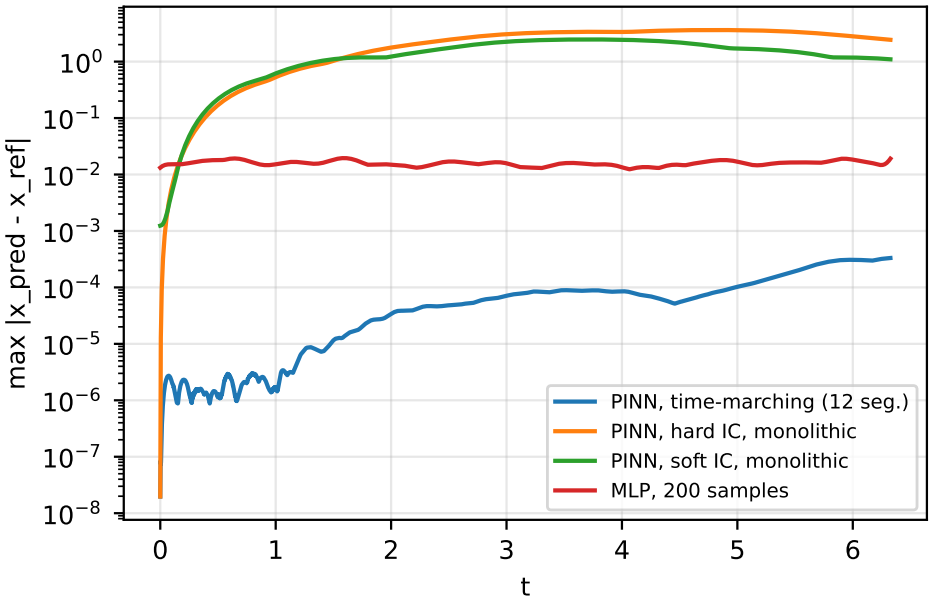



Figure 2: Pointwise maximum position error against time. The time-marching PINN stays below $10^{-3}$ on the full period; the monolithic PINNs and the supervised MLP stay near $10^{0}$ or $10^{-2}$.

### 5.3 Extrapolation beyond the training window

Evaluating the figure-eight PINN on $[T, 2T]$ gives a maximum position error of 23.6, far larger than the error on the trained window. The network has no information about times beyond $T$ (each segment is trained only on $[0, L]$), and the network extrapolates poorly. The method therefore represents a trajectory on the window it was trained on and nothing more. Periodicity can be exploited by evaluating on $[0, T]$ modulo $T$, which we did not do here because it would hide the failure. The Lagrange triangle shows the same behaviour (10.2 on $[T, 2T]$).

## 6 Discussion

**What the results support.** Within its training window, and for the three tested initial conditions, the time-marching PINN reproduces the reference trajectory and conserves energy to $10^{-4}$ relative accuracy. The method needs no trajectory data, and its accuracy on the figure-eight exceeds that of a supervised MLP with 200 exact samples by a factor of about 60.

**Limitations.**

1. *No generalisation over initial conditions.* Each orbit is trained from scratch, at a cost of about 12 minutes. A parametric network $N_\theta(t, y^0)$ is the natural extension, but it was not built or tested here; the claim is restricted to fixed initial conditions.

2. *No extrapolation.* Predictions outside the training window are not valid (Section 5.3).

3. *Close encounters and chaos.* The random-IC test was filtered to avoid near collisions over $[0, 2]$. Long-horizon chaotic orbits and close encounters have not been tested, and both are expected to be hard for this method, which has no mechanism to resolve singular accelerations.

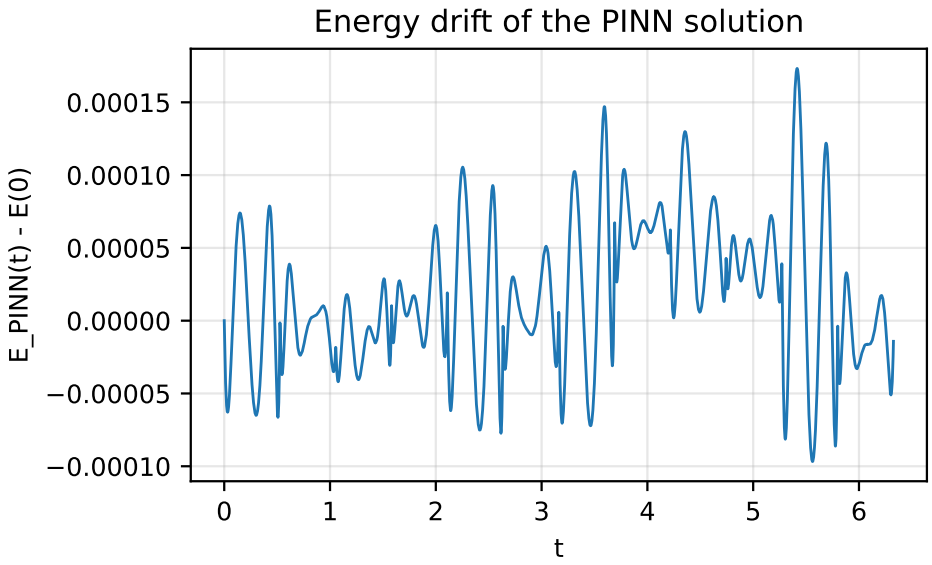



Figure 3: Energy of the PINN solution minus its initial value, computed from the network's positions and velocities. The drift stays below $2 \times 10^{-4}$ in absolute value over one period.

4. *Equal masses only.* Both periodic tests use equal masses. The residual code accepts general masses, but unequal-mass problems were not tested.

5. *Single seed for the baselines.* The baseline numbers come from one seed each and are not averaged.

## 7 Conclusion

A PINN with a hard initial-condition ansatz and time marching reproduces periodic three-body orbits to a maximum error of about $3 \times 10^{-4}$ on the time window it is trained on, without any trajectory data. Monolithic PINNs and a supervised MLP with 200 samples are both substantially less accurate on the same problem. The method does not extrapolate and has not been tested on chaotic orbits or parametric families, which are the next steps.

## Reproducibility

Code and results are in a single project directory. `nbody.py` contains the physics, the reference integrator, and the initial conditions. `pinn.py` contains the ansatz, the residual, time marching and the supervised baseline. `run_orbit.py` trains and evaluates the time-marching PINN on a periodic orbit, `run_baselines.py` the baselines, `run_random.py` the random-IC test, and `make_figures.py` the figures. Training logs for the convergence study are in `results/exploration/`. Exact commands are given with the code. Experiments were run with PyTorch 2.14 on CUDA.

## References

[1] V. Szebehely. *Theory of Orbits: The Restricted Problem of Three Bodies*. Academic Press, 1967.

[2] A. Chenciner and R. Montgomery. A remarkable periodic solution of the three-body problem in the case of equal masses. *Annals of Mathematics* 152(3):881–901, 2000.

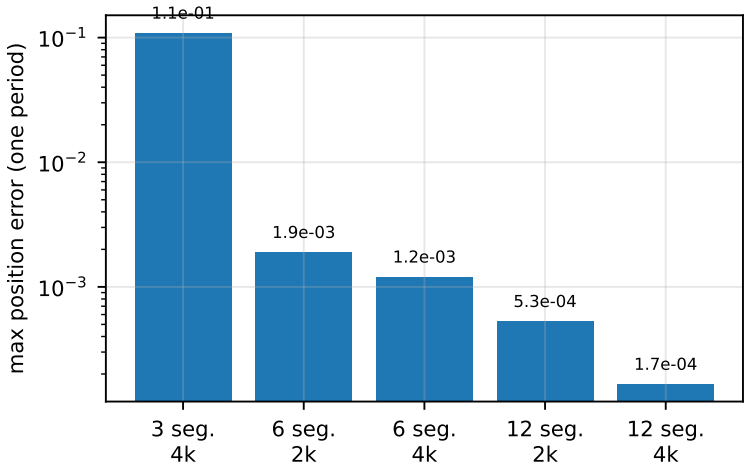



Figure 4: Maximum position error on the figure-eight against segment count and Adam budget (Table 2).


[3] J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics* 6(1):19–26, 1980.

[4] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272, 2020.

[5] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* 378:686–707, 2019.

[6] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, and M. W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems* 34, 2021.

[7] S. Wang, Y. Teng, and P. Perdikaris. Understanding and mitigating gradient pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing* 43(5):A3055–A3081, 2021.

[8] S. Wang, X. Yu, and P. Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics* 449:110768, 2022.

[9] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.